

SOLID Principles

Five foundational principles of object-oriented design, explained with Java examples

SOLID is a set of five design principles that help you write maintainable, flexible, and scalable object-oriented code. Each principle addresses a specific way that code can become rigid, fragile, or hard to change — and each comes with a concrete fix.

1. Single Responsibility Principle (SRP)

A class should have only one reason to change — meaning it should have only one job or responsibility.

Bad example

```
class Invoice {
    private double amount;

    public double calculateTotal() {
        return amount * 1.14; // adding tax
    }

    // Violates SRP - this class now handles persistence too
    public void saveToDatabase() {
        // database connection logic
        System.out.println("Saving invoice to DB...");
    }

    // And now it handles printing too!
    public void printInvoice() {
        System.out.println("Invoice total: " + calculateTotal());
    }
}
```

This Invoice class has three reasons to change: if tax calculation changes, if the database changes, or if the print format changes.

Good example

```
class Invoice {
    private double amount;

    public double calculateTotal() {
        return amount * 1.14;
    }
}

class InvoiceRepository {
    public void save(Invoice invoice) {
        System.out.println("Saving invoice to DB...");
    }
}

class InvoicePrinter {
    public void print(Invoice invoice) {
```

```

        System.out.println("Invoice total: " + invoice.calculateTotal());
    }
}

```

Now each class has exactly one job. Changing how invoices are stored doesn't touch Invoice or InvoicePrinter.

2. Open/Closed Principle (OCP)

Software entities should be open for extension, but closed for modification. You should be able to add new behavior without changing existing, tested code.

Bad example

```

class DiscountCalculator {
    public double calculate(String customerType, double amount) {
        if (customerType.equals("REGULAR")) {
            return amount * 0.9;
        } else if (customerType.equals("VIP")) {
            return amount * 0.8;
        }
        // Every new customer type means editing this method again
        return amount;
    }
}

```

Every time you add a new customer type, you have to modify this class and risk breaking existing logic.

Good example

```

interface DiscountStrategy {
    double applyDiscount(double amount);
}

class RegularDiscount implements DiscountStrategy {
    public double applyDiscount(double amount) { return amount * 0.9; }
}

class VipDiscount implements DiscountStrategy {
    public double applyDiscount(double amount) { return amount * 0.8; }
}

class DiscountCalculator {
    public double calculate(DiscountStrategy strategy, double amount) {
        return strategy.applyDiscount(amount);
    }
}

```

To add a new discount type (e.g., StudentDiscount), you just create a new class implementing DiscountStrategy — you never touch DiscountCalculator.

3. Liskov Substitution Principle (LSP)

Subtypes must be substitutable for their base types without breaking the correctness of the program. If S is a subtype of T, objects of type T should be replaceable with objects of type S without altering the

program's behavior.

Classic bad example (the Rectangle/Square problem)

```
class Rectangle {
    protected int width, height;

    public void setWidth(int w) { this.width = w; }
    public void setHeight(int h) { this.height = h; }
    public int getArea() { return width * height; }
}

class Square extends Rectangle {
    @Override
    public void setWidth(int w) {
        this.width = w;
        this.height = w; // forces height too, breaking expectations
    }

    @Override
    public void setHeight(int h) {
        this.width = h;
        this.height = h;
    }
}

Rectangle r = new Square();
r.setWidth(5);
r.setHeight(10);
System.out.println(r.getArea()); // Expected 50, but gets 100
```

Anyone using `Rectangle` expects that setting width and height independently gives predictable area — `Square` breaks that contract, violating LSP.

Better approach

Don't force `Square` to inherit from `Rectangle` if it can't honor the same behavior. Instead, use a common abstraction:

```
interface Shape {
    int getArea();
}

class Rectangle implements Shape {
    private int width, height;
    public Rectangle(int w, int h) { width = w; height = h; }
    public int getArea() { return width * height; }
}

class Square implements Shape {
    private int side;
    public Square(int s) { side = s; }
    public int getArea() { return side * side; }
}
```

4. Interface Segregation Principle (ISP)

Clients should not be forced to depend on interfaces they don't use. Prefer many small, specific interfaces over one large, general-purpose one.

Bad example

```
interface Worker {
    void work();
    void eat();
}

class HumanWorker implements Worker {
    public void work() { System.out.println("Working..."); }
    public void eat() { System.out.println("Eating lunch..."); }
}

class RobotWorker implements Worker {
    public void work() { System.out.println("Working..."); }
    public void eat() {
        // Robots don't eat! Forced to implement a useless method
        throw new UnsupportedOperationException();
    }
}
```

Good example

```
interface Workable {
    void work();
}

interface Eatable {
    void eat();
}

class HumanWorker implements Workable, Eatable {
    public void work() { System.out.println("Working..."); }
    public void eat() { System.out.println("Eating lunch..."); }
}

class RobotWorker implements Workable {
    public void work() { System.out.println("Working..."); }
    // No need to implement eat() at all
}
```

Now each class only implements the behavior that's actually relevant to it.

5. Dependency Inversion Principle (DIP)

High-level modules should not depend on low-level modules; both should depend on abstractions.

Also: abstractions should not depend on details — details should depend on abstractions.

Bad example

```
class MySQLDatabase {
    public void save(String data) {
        System.out.println("Saving to MySQL: " + data);
    }
}

class UserService {
```

```

private MySQLDatabase database = new MySQLDatabase(); // tightly coupled

public void saveUser(String user) {
    database.save(user);
}
}

```

If you ever want to switch to PostgreSQL or MongoDB, you must modify UserService directly — it's tightly coupled to a specific implementation.

Good example

```

interface Database {
    void save(String data);
}

class MySQLDatabase implements Database {
    public void save(String data) {
        System.out.println("Saving to MySQL: " + data);
    }
}

class MongoDBDatabase implements Database {
    public void save(String data) {
        System.out.println("Saving to MongoDB: " + data);
    }
}

class UserService {
    private Database database;

    // Dependency is injected, not hardcoded
    public UserService(Database database) {
        this.database = database;
    }

    public void saveUser(String user) {
        database.save(user);
    }
}

UserService service = new UserService(new MongoDBDatabase());
service.saveUser("Mohamed");

```

UserService now depends only on the Database abstraction. Swapping databases means creating a new implementation — zero changes to UserService.

Quick Summary

Principle	Core Idea
SRP	One class, one responsibility
OCP	Extend behavior without modifying existing code

LSP	Subclasses must honor the parent's contract
ISP	Small, focused interfaces over bloated ones
DIP	Depend on abstractions, not concrete implementations

These principles work well together — for instance, DIP often relies on interfaces created via ISP, and OCP is usually achieved by leveraging polymorphism enabled by good LSP design. Frameworks like Spring rely heavily on DIP through dependency injection.